



AWS GenAI POC

Motadata

> 4 AWS Programs



> 9 AWS Competencies



> 19 AWS Service Delivery Programs



Revision History



Version	Date	Description	Author	Reviewer
v1.0	12-08-2026	Initial Draft	Vynkatesh S...	Aditya Ka...
v2.0	22-07-2026	Updated the description of the investigator agent in the document.	Vynkatesh S...	Aditya Ka...
v3.0	06-08-2026	Updated about the RDS PostgreSQL Setup	Vynkatesh S...	Aditya Ka...



Table of Contents

Introduction	4
Overview of the Setup	4
ECS Cluster Setup	5
Amazon Bedrock AgentCore Setup	9
S3 Bucket Setup	13
RDS PostgreSQL Setup	14
CloudFront and Cognito Setup	16
Code Change and Deployment Process	17
Tenant Account Management	21



Introduction

This Knowledge Transfer (KT) document is a comprehensive guide for understanding and managing the AWS infrastructure setup for the Motadata AI-Powered RCA Agent. It aims to provide detailed insights into the various AWS services and configurations used in the deployment. The document is designed to empower team members, administrators, and relevant stakeholders with the knowledge to operate and maintain the deployed infrastructure effectively.

Overview of the Setup

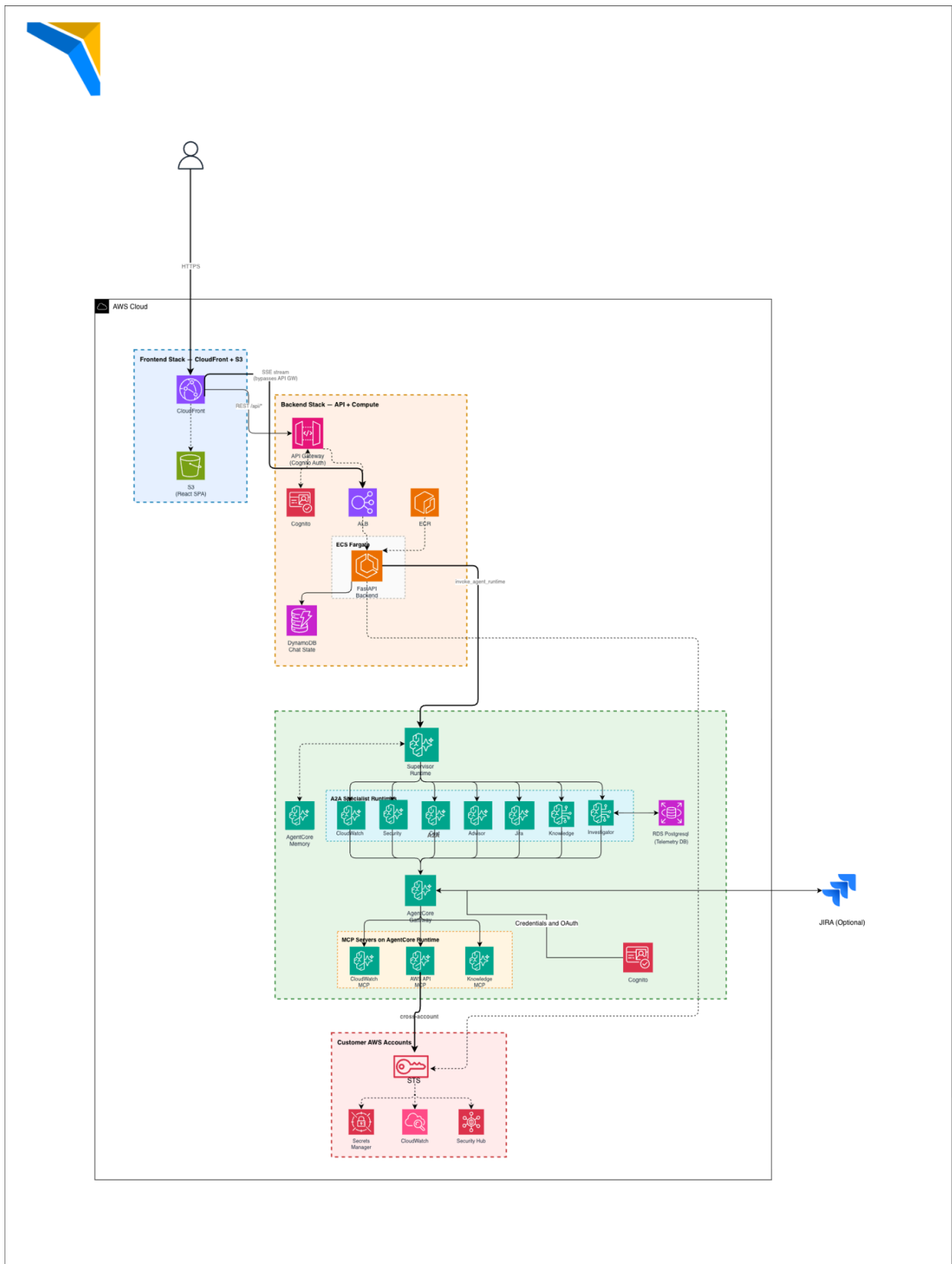
The Motadata GenAI Proof of Concept (POC) implements a containerized, cloud-native multi-agent AI architecture deployed on AWS using Amazon Bedrock AgentCore and LangGraph. The system is designed to support automated Root Cause Analysis by correlating telemetry data across metrics, logs, and traces through a natural language chat interface.

The solution exposes a secure web interface through Amazon CloudFront backed by AWS WAF. API requests are routed to an ECS Fargate service running the FastAPI backend, which acts as a thin orchestration layer handling authentication, session management, and delegation to AI agents. Static frontend assets are served from an S3 origin bucket.

The AI layer runs entirely on Amazon Bedrock AgentCore with a Supervisor agent orchestrating 7 specialist agents via Agent-to-Agent (A2A) protocol. Each specialist handles a specific domain – CloudWatch monitoring, Security Hub findings, Cost Explorer analysis, Trusted Advisor recommendations, Jira ticket management, AWS documentation search, and telemetry-based RCA investigation. Tool access is provided through an AgentCore Gateway with registered MCP (Model Context Protocol) servers.

Multi-tenant isolation is enforced at every layer through Cognito JWT authentication, AgentCore Memory namespacing per user, and STS AssumeRole for per-tenant AWS account access. The RCA Investigator agent queries a tenant-scoped PostgreSQL database containing OpenTelemetry-format telemetry data to perform deep causal analysis with evidence correlation.

The system is deployed in a single AWS account in the ap-south-1 (Mumbai) region, with CloudFront and WAF operating globally from us-east-1.





ECS Cluster Setup

The Motadata RCA Agent backend is deployed on Amazon ECS Fargate within the ECS cluster named poc-cluster in the ap-south-1 region. The cluster runs a single ECS service named poc-backend-service with a desired count of 2 Fargate tasks for high availability. Each task runs with 512 CPU units and 1024 MB memory, using the latest revision of the task definition poc-backend-task. The service is deployed in private subnets (subnet-042bf0a08c81d9bc0, subnet-0757cf15768a0a5c0, subnet-0ded7be4062c01f27) within the VPC and configured with auto-scaling based on CPU target tracking (scale-out at 70%, scale-in at 63%).

The screenshot displays the AWS Management Console for the Amazon Elastic Container Service (ECS). The breadcrumb navigation shows the path: Amazon Elastic Container Service > Clusters > poc-cluster > Services > poc-backend-service > Health. The left-hand navigation pane includes links for Favorites and recents, Express Mode, Clusters, Namespaces, Task definitions, Daemon task definitions, Account settings, AWS Resource Explorer, AWS Batch, Amazon ECR, Repositories, Online learning workshop, Documentation, Discover products, and Subscriptions. The main content area is titled 'poc-backend-service' and includes a 'Service overview' section with the following details: Status is 'Active', Tasks (2 Desired) are '2 running | 0 pending', Task definition is 'revision poc-backend-task:17', and Deployment status is 'Success'. Below this is a 'Health and metrics' section with tabs for Tasks, Logs, Deployments, Events, Configuration and networking, Action logs - new, Service auto scaling, and Event history. The 'Status' section shows the Service name 'poc-backend-service', Service ARN 'arn:aws:ecs:ap-south-1:001961766007...', Deployments current state '2 Completed tasks', and Created at 'June 17, 2026, 12:03 (UTC+5:30)'. A 'Load balancer target health' table is also present:

Container name:port	Target group	Resource type	Target health	Load balancer	Load balancer type
backend:8000	POCAss-ALBHT-1KWXBJP...	Customer managed	2 Healthy 0 Unhealthy	poc-alb	Application Load Balancer

The 'Health' section at the bottom shows 'ECS metrics' and 'Container insights metrics' tabs, with a 'View Container Insights' link. It also includes an 'Alarm recommendations' section and a time range selector set to '1h'.

ECR Setup

Container images for the backend application are stored in Amazon Elastic Container Registry (ECR) under a private repository in ap-south-1. The repository 001961766007.dkr.ecr.ap-south-1.amazonaws.com/poc-backend stores the FastAPI application image built from the project's Dockerfile. Updated Docker images are built locally with the linux/amd64 platform flag, tagged, and pushed to this repository. When a new image is pushed, the ECS service is updated with a force-new-deployment command to pull the latest image.



The screenshot shows the AWS Elastic Container Service console for the 'poc-backend-service'. The left sidebar contains navigation links for Amazon Elastic Container Service, Favorites and recents, Express Mode, Clusters, Namespaces, Task definitions, Daemon task definitions, Account settings, AWS Resource Explorer, AWS Batch, Amazon ECR, Repositories, Online learning workshop, Documentation, Discover products, and Subscriptions. The main content area shows the 'poc-backend-service' overview with tabs for Health and metrics, Tasks, Logs, Deployments, Events, Configuration and networking, Action logs - new, Service auto scaling, and Event history. The 'Health and metrics' tab is active, showing the service status as 'Active' with 2 tasks desired (2 running, 0 pending). The 'Status' section shows the service name 'poc-backend-service', Service ARN 'arn:aws:ecs:ap-south-1:0019617660...', Deployments current state as '2 Completed tasks', and Created at 'June 17, 2026, 12:03 (UTC+5:30)'. The 'Load balancer target health' table shows the container name 'port', Target group 'POCAss-ALBHt-1KWX8JPMJTbQ', Resource type 'Customer managed', Target health '2 Healthy, 0 Unhealthy', Load balancer 'poc-alb', and Load balancer type 'Application Load Balancer'. The 'Health' section shows ECS metrics and Container insights metrics, with a 'View Container Insights' link. The bottom of the page has a 'Tell us what you think' link and a 'Learn more about Container Insights metrics' link.

Application Load Balancer (ALB) Setup

Inbound API traffic is managed by an internal Application Load Balancer named poc-alb in the ap-south-1 region. The ALB DNS endpoint is poc-alb-1735024846.ap-south-1.elb.amazonaws.com. It is associated with security group sg-0c795fe63c351e3b3 and forwards traffic to the target group POCAss-ALBHt-1KWX8JPMJTbQ on port 8000.

CloudFront routes all /api/* requests to this ALB as a custom origin with HTTP-only origin protocol (port 80). The ALB performs health checks on the /health endpoint with 30-second intervals and 10-second timeouts. The ALB is placed in private subnets and is not directly accessible from the internet – all external traffic flows through CloudFront.



The screenshot displays the AWS Management Console interface for an Application Load Balancer (ALB) named 'poc-alb'. The left sidebar shows the navigation menu with categories like EC2, Instances, Images, Elastic Block Store, and Network & Security. The main content area shows the 'Details' tab for the ALB, which is in an 'Active' state. Key details include the VPC (vpc-0d4098f4a71dd9fb9), Availability Zones (ap-south-1a, ap-south-1c, ap-south-1b), and the Load balancer IP address type (IPv4). The 'Listeners and rules' section is also visible, showing one listener.

Task Definition Containers

Each ECS task runs three containers:

- **Backend :**
 - The FastAPI application serves the chat API on port 8000. Built from the project Dockerfile at backend/Dockerfile using Python 3.11 Alpine base.
- **Adot-collector :**
 - AWS Distro for OpenTelemetry (ADOT) collector (v0.40.0) running as a sidecar. Receives OpenTelemetry traces on ports 4317 (gRPC) and 4318 (HTTP) and forwards them to AWS X-Ray. Configured via the AOT_CONFIG_CONTENT environment variable with a health check extension on port 13133.

Key environment variables configured on the backend container include `AWS_REGION` (ap-south-1), `COGNITO_USER_POOL_ID` (ap-south-1_XoMzeIIsf), `SUPERVISOR_RUNTIME_ARN` (the AgentCore supervisor ARN), `MEMORY_ID` (poc_msp_memory-LxXr4nGE6g), `GATEWAY_URL` (the MCP gateway endpoint), `OTEL_ENABLED` (true), and `DATABASE_URL` (PostgreSQL connection string for the RCA telemetry database).



Amazon Bedrock AgentCore Setup

The agent orchestration layer is deployed entirely on Amazon Bedrock AgentCore, consisting of 11 runtimes, 1 gateway, and 1 memory resource. All runtimes are managed via the AgentCore CLI from the CDK project located at `agentcore-deploy/motadatamsp/`.

Runtimes :

The system deploys 11 AgentCore Runtimes — 1 Supervisor, 7 specialist A2A agents, and 3 MCP tool servers. All runtimes use `networkMode: PUBLIC` with HTTP protocol on port 8080, and each has `enableOtel: true` for observability.

- **Supervisor Runtime**

- `(motadatamsp_supervisor_langgraph-9KnVEoGxwe)` is the entry point for all user queries. It runs a `LangGraph create_react_agent` with `Claude Sonnet 4.5 (global.anthropic.claude-sonnet-4-5-20250929-v1:0)` and has 7 delegation tools that invoke specialist agents via A2A protocol. The supervisor loads conversation memory, classifies intent using an SRE-methodology prompt, and routes to the appropriate specialist.

- **CloudWatch Agent**

`(motadatamsp_cloudwatch_a2a-H5RcGb9DcS)` queries live CloudWatch alarms, metrics, and log groups for the selected tenant's AWS account using STS temporary credentials read from Secrets Manager.

- **Security Agent** `(motadatamsp_security_a2a-IXIDFxBpup)` retrieves Security Hub findings, CVEs, and compliance status from the tenant's account using the same STS credential injection pattern.

- **Cost Agent** : `(motadatamsp_cost_a2a-wzbLX73tnc)` queries AWS Cost Explorer for spending patterns and optimization opportunities.

- **Advisor Agent** : `(motadatamsp_advisor_a2a-rLgT026HbV)` retrieves Trusted Advisor recommendations across cost, security, performance, and fault tolerance pillars.



- **Jira Agent :** (motadatamsp_jira_a2a-F9Z6ao4iK8) manages Jira tickets — creating, querying, and updating issues for incident tracking.
- **Knowledge Agent :** (motadatamsp_knowledge_a2a-HH7Is0DX6i) searches AWS documentation and best practices to support investigation recommendations.
- **Investigator Agent :** (motadatamsp_investigator_a2a-ihVIFH4B57) is the core RCA engine. It connects to a PostgreSQL telemetry database using LangChain's SQLDatabaseToolkit and performs multi-step SQL-based investigation across otel_metrics, otel_logs, and otel_spans tables with a recursion limit of 30. It uses Claude Haiku 4.5 and filters all queries by tenant_id for multi-tenant isolation.
- **MCP Servers :** (CloudWatch MCP, AWS API MCP, AWS Knowledge MCP) provide tool access through the AgentCore Gateway using SigV4 + OAuth authentication.

The screenshot displays the Amazon Bedrock AgentCore console interface. The left sidebar contains navigation options: Discover, Build, Test, Assess, and Docs and resources. The main content area is titled 'Runtime' and includes sections for 'Build a runtime', 'Host in runtime', and 'Assess performance using observability'. Below these, there is a 'Runtime resources (12)' table listing various agents and their status.

Name	Status	Managed by	Description	Latest version	Last updated (UTC+05...)
motadatamsp_cloudwat...	Ready	-	-	Version 22	July 22, 2026, 10:36
motadatamsp_security...	Ready	-	-	Version 23	July 22, 2026, 10:36
motadatamsp_cloudwat...	Ready	-	AgentCore Runtime: mot...	Version 12	July 22, 2026, 10:34
motadatamsp_aws_api...	Ready	-	AgentCore Runtime: mot...	Version 15	July 22, 2026, 10:34
motadatamsp_aws_kno...	Ready	-	AgentCore Runtime: mot...	Version 11	July 22, 2026, 10:34
motadatamsp_supervis...	Ready	-	-	Version 24	July 20, 2026, 10:39
motadatamsp_investiga...	Ready	-	-	Version 24	July 19, 2026, 03:17
motadatamsp_advisor...	Ready	-	AgentCore Runtime: mot...	Version 14	July 15, 2026, 12:24
motadatamsp_jira_a2a	Ready	-	AgentCore Runtime: mot...	Version 12	July 15, 2026, 12:24
motadatamsp_knowled...	Ready	-	AgentCore Runtime: mot...	Version 14	July 15, 2026, 12:24

Gateway

The AgentCore Gateway named msp-assistant-gateway-hygxaasfuh provides a unified MCP endpoint for tool access. It has 3 registered



targets: the CloudWatch MCP server (providing CloudWatch Logs, Metrics, and Alarms tools), the AWS API MCP server (providing general AWS API access), and the AWS Knowledge MCP server (providing documentation search). Specialist agents connect to the gateway via SigV4-signed requests at the endpoint <https://msp-assistant-gateway-hygxaasfuh.gateway.bedrock-agentcore.ap-south-1.amazonaws.com/mcp>.

The screenshot shows the Amazon Bedrock AgentCore console. The left sidebar contains navigation links for Discover, Build, Test, Assess, and Docs and resources. The main content area is titled 'How it works' and includes three sections: 'Create a gateway', 'Create configuration bundles for your agentic application', and 'Assess performance using observability'. Below these sections, the 'Gateways' tab is selected, showing a table of gateways. The table has columns for Gateway name, Description, Status, and Creation time. Two gateways are listed: 'poc-msp-gateway' and 'msp-assistant-gateway', both with a status of 'Ready'.

Gateway name	Description	Status	Creation time
poc-msp-gateway	-	Ready	June 17, 2026, 14:33 (UTC+05:30)
msp-assistant-gateway	-	Ready	June 12, 2026, 13:27 (UTC+05:30)

Memory

AgentCore Memory resource poc_msp_memory-LxXr4nGE6g provides conversational persistence with two extraction strategies configured:

- **SemanticFacts :**

- Extracts factual statements from conversations (e.g., "The user's production account is 123456789") and stores them as retrievable semantic facts that persist across sessions.

- **SessionSummaries :**

- Generates concise summaries of completed sessions capturing key topics, decisions, and actions taken.

Memory is namespaced by actorId (derived from the Cognito user's sub claim), ensuring complete isolation between users. The namespace pattern is /strategy/{strategy_id}/actor/{actorId}/.

Foundation Models



Agent	Model	Purpose
Supervisor Agent	Anthropic Claude Sonnet 4.5	Intent classification, query routing, multi-agent orchestration, and response synthesis.
CloudWatch Agent	Anthropic Claude Haiku 4.5	Interpreting CloudWatch alarms, metrics, and log data.
Security Agent	Anthropic Claude Haiku 4.5	Analyzing Security Hub findings and compliance status.
Cost Agent	Anthropic Claude Haiku 4.5	Cost Explorer data interpretation and optimization recommendations.
Advisor Agent	Anthropic Claude Haiku 4.5	Summarizing Trusted Advisor checks and recommendations.
Jira Agent	Anthropic Claude Haiku 4.5	Jira ticket creation, querying, and updates.
Knowledge Agent	Anthropic Claude Haiku 4.5	AWS documentation search and retrieval.
Investigator Agent	Anthropic Claude Haiku 4.5	SQL-based telemetry investigation, multi-step RCA reasoning, and evidence correlation across metrics, logs, and traces.

Multi-Tenant Isolation

The system supports multiple customer accounts (tenants) with complete data separation. When a user selects a customer account from the dropdown in the UI, all agent queries are automatically scoped to that tenant's environment.

Tenant isolation is implemented across five layers:

- **Authentication** — Users log in via Cognito. The JWT token's sub claim identifies the user and scopes all memory and session data to that user only.



- **AWS Account Access** –
 - Each tenant's AWS resources (CloudWatch, Security Hub, Cost Explorer) are accessed using temporary STS credentials stored in Secrets
 - Manager under `mvp-credentials/{tenant_name}`. Credentials expire after 1 hour and are auto-refreshed by the backend.
- **Telemetry Database** – The Investigator agent filters every SQL query by `tenant_id`. Each tenant's metrics, logs, and traces are stored in the same PostgreSQL tables but isolated by this column.
- **Memory** – AgentCore Memory namespaces conversations per user using the pattern `/strategy/{id}/actor/{actorId}/`. No user can see another user's chat history or extracted facts.
- **Runtime** – Each AgentCore invocation runs in a separate microVM. No state is shared between sessions.

To add a new tenant, create the credential secret in Secrets Manager and optionally tag the RDS instance with the tenant name. No code changes are required.

S3 Bucket Setup

The Motadata RCA Agent uses S3 buckets for frontend hosting and telemetry asset storage:

Pocassistantfrontendstack-frontends3bucket834ef3a2-3fwqppbpvcht :

- Region: ap-south-1
- Purpose: React 18 SPA hosting via CloudFront Origin Access Identity.
- Files: `index.html`, `assets/js/`, `assets/css/`, `vite.svg`.
- Access: CloudFront OAI only (public access blocked).
- Integration: CloudFront distribution `E2KKBKVV0SMTV0`.

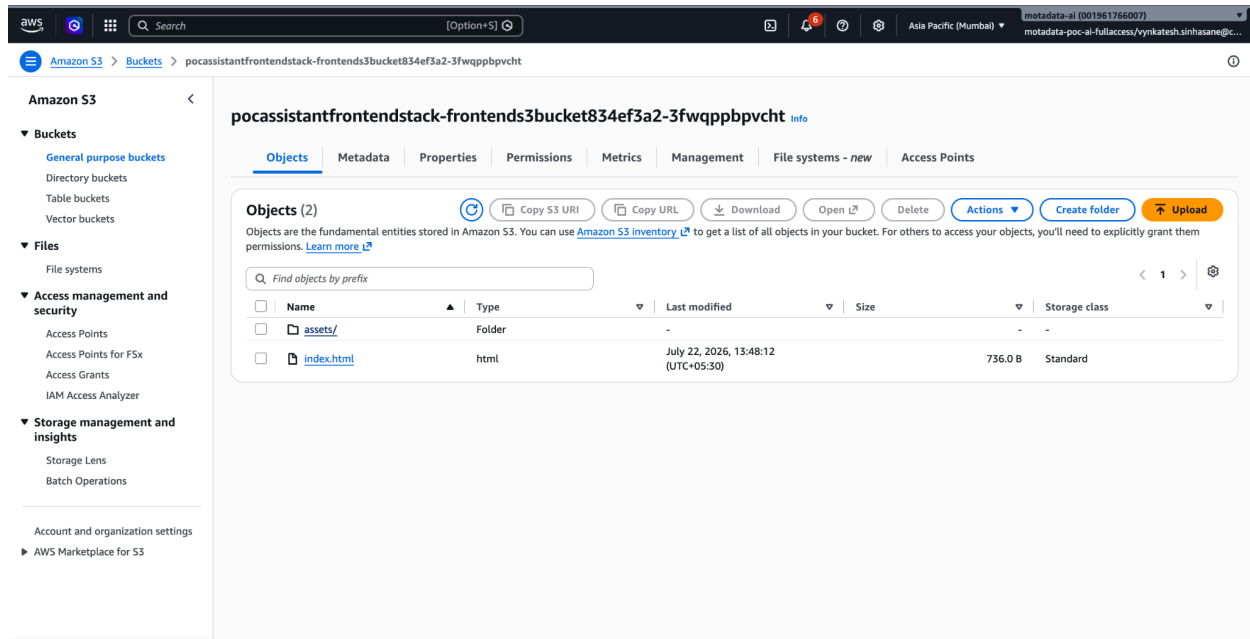
Poc-agentcore-assets-001961766007-ap-south-1 :

- Region: ap-south-1
- Purpose: Telemetry data storage and agent deployment artifacts.
- Files: Tenant telemetry CSVs, agent packages, runbooks.
- Access: ECS task role, AgentCore execution roles.
- Integration: Agent runtimes, telemetry ingestion.

Security (All Buckets):



- Public access blocked (PublicAccessBlock enabled)
- SSE-S3 encryption (AES-256) at rest
- IAM least-privilege bucket policies
- Versioning enabled



RDS PostgreSQL Setup

The RCA Investigator agent queries a PostgreSQL database containing OpenTelemetry-format telemetry data. The RDS instance `rca-scenario-db` runs PostgreSQL 16.14 on a `db.t3.micro` instance class with 20 GB gp3 storage in `ap-south-1`. The endpoint is `rca-scenario-db.cng0ieo6guoi.ap-south-1.rds.amazonaws.com` on port 5432. The instance is publicly accessible (POC configuration) with security group `sg-056b27a95cb60d3a1` allowing inbound on port 5432 from `0.0.0.0/0`. Database credentials are stored in AWS Secrets Manager under the keys `rca-db/default` and `rca-db/poc_account`. The database contains three tables used for RCA investigation:

- **Otel_metrics (12,120 rows per tenant) :**
 - Time-series metrics including `checkout_success_rate`, `dns_query_latency_p99_ms`, `kernel_rx_queue_latency_ms`, `payment_connection_acquire_ms`, and `irq_affinity_numa_mismatch`. Each row contains `tenant_id`, `ts`



(timestamp), metric_name, value, and labels (JSONB with dimensions like region, cluster, node_pool, pod).

- **otel_logs (286 rows per tenant) :**
 - Application and kernel logs from streams payment_service_logs, checkout_service_logs, kernel_logs, and infra_change_logs. Each row contains tenant_id, ts, stream, level, logger, message, and attributes (JSONB).
- **otel_spans (840 rows per tenant) :**
 - Distributed traces covering 84 complete request traces across 9 services (checkout, pricing, inventory, promotion, order, payment, token, fraud, payment-provider). Each row contains tenant_id, trace_id, span_id, parent_span_id, service_name, span_name, span_kind, start_time, end_time, duration_ms, and attributes (JSONB).

All tables include a tenant_id column and indexes on (tenant_id, metric_name, ts), (tenant_id, stream, ts), and (tenant_id, trace_id) for efficient tenant-scoped queries.

The screenshot displays the AWS Management Console for an Amazon RDS instance named 'rca-scenario-db'. The console shows the instance's status as 'Available' and its configuration details, including the engine (PostgreSQL), region (Asia Pacific (Mumbai)), and class (db.t3.micro). The 'Connectivity & security' tab is selected, showing options for connecting using code snippets, CloudShell, or endpoints. The 'Internet access gateway' is disabled, and 'IAM Authentication' is also disabled. The 'Programming language' is set to 'PSQL (macOS)' and the 'Endpoint type' is 'Instance endpoint'. The 'Connection steps' section provides the following commands to connect to the database:

```
1 curl -o global-bundle.pem https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem
2 export RDSHOST="rca-scenario-db.cng0ieo6guoi.ap-south-1.rds.amazonaws.com"
3 psql "host=$RDSHOST port=5432 dbname=postgres user=rcaadmin sslmode=verify-full sslrootcert=./global-bundle.pem"
```

CloudFront and Cognito Setup

CloudFront



The application is served through Amazon CloudFront distribution E2KKBKVV0SMTV0 at <https://d2e9bh1pq1a0sj.cloudfront.net>. The distribution has two origins:

S3 Origin (default) :

Serves React frontend static assets with CachingOptimized policy. Custom error responses for 403/404 return /index.html with HTTP 200 to support SPA client-side routing.

ALB Origin (/api/*) :

Routes all API requests to the internal ALB with CachingDisabled policy (no caching on API responses) and AllViewerExceptHostHeader origin request policy. Read timeout is set to 60 seconds for streaming responses.

AWS WAF (poc-waf, ARN:arn:aws:wafv2:us-east-1:001961766007:global/webacl/poc-waf/e80736e1-d8d0-43a9-9ef3-8d623923d0bb) is attached to the distribution for SQL injection, XSS, and rate limiting protection.

Cognito

Amazon Cognito User Pool ap-south-1_XoMzeIISf provides user authentication with web client ID 7bs3ae73cnblm9o44io7vg4tk3. Users authenticate via email/password and receive JWT tokens (1-hour access, 30-day refresh). The backend validates JWTs using the Cognito



JWKS endpoint on every API request. The user's sub claim becomes the actorId for AgentCore Memory namespacing.

Code Change and Deployment Process

How to Make Code Changes and Redeploy — Backend

1. Make Code Changes

Navigate to the project root directory (which contains the backend/Dockerfile file):

```
cd
```

```
/Applications/Comprinno/Client/Motadata/msp-ops-langgraph
```

Key Backend Files

- backend/app/main.py — FastAPI application entry point
- backend/app/api/routes.py — API endpoints (chat, accounts, health)
- backend/app/core/langgraph_client.py — Thin client for AgentCore invocation
- backend/app/core/workspace_context.py — Tenant credential management
- backend/app/core/telemetry.py — OpenTelemetry instrumentation
- backend/requirements.txt — Python dependencies

Make the required code changes and save the files.

2. Authenticate Docker with Amazon ECR

For macOS/Linux:

```
aws ecr get-login-password --region ap-south-1 --profile  
motadata | \
```

```
docker login --username AWS --password-stdin \  
001961766007.dkr.ecr.ap-south-1.amazonaws.com
```



3. Build the Docker Image

```
docker build --platform linux/amd64 \  
-t \  
001961766007.dkr.ecr.ap-south-1.amazonaws.com/poc-backend: \  
latest \  
-f backend/Dockerfile .
```

Note: The build context is the project root (not backend/) because the Dockerfile copies both the backend/ and agents/ directories.

4. Push the Image to Amazon ECR

```
docker push \  
001961766007.dkr.ecr.ap-south-1.amazonaws.com/poc-backend:latest
```

5. Update the ECS Service

```
aws ecs update-service \  
--cluster poc-cluster \  
--service poc-backend-service \  
--force-new-deployment \  
--region ap-south-1 \  
--profile motadata
```

The deployment uses a rolling update strategy with a maximum of 200% and a minimum of 50% healthy tasks.

Monitor the deployment status:

```
aws ecs describe-services \  
--cluster poc-cluster \  
--services poc-backend-service \  
--region ap-south-1 \  
--profile motadata \  
--query
```

```
'services[0].deployments[*].[status,runningCount,rolloutState]'
```

Wait until the PRIMARY deployment displays:

rolloutState: COMPLETED

and the previous deployment has been fully drained.

How to Make Code Changes and Redeploy — AgentCore Agents

1. Make Code Changes

Agent source code is located in the agents/ directory.

Agent Runtime Files



- agents/runtime/supervisor_runtime.py – Supervisor agent (routing and orchestration)
- agents/runtime_cloudwatch_langgraph/cloudwatch_a2a_runtime.py – CloudWatch specialist
- agents/runtime_security_langgraph/security_a2a_runtime.py – Security specialist
- agents/runtime_cost_langgraph/cost_a2a_runtime.py – Cost specialist
- agents/runtime_advisor_langgraph/advisor_a2a_runtime.py – Advisor specialist
- agents/runtime_jira_langgraph/jira_a2a_runtime.py – Jira specialist
- agents/runtime_knowledge_langgraph/knowledge_a2a_runtime.py – Knowledge specialist
- agents/runtime_investigator_langgraph/investigator_a2a_runtime.py – RCA Investigator

Each agent also contains:

- aws_tools.py – Local boto3 tool implementations
- gateway_client.py – MCP Gateway connectivity

Make the required changes and save the files.

2. Deploy the Agents

```
cd agentcore-deploy/motadatamsp
eval $(aws configure export-credentials --profile motadata
--format env)
agentcore deploy --yes
```

This command:

- Builds container images for all modified agents.
- Pushes the images to Amazon ECR.
- Updates the AgentCore CDK stack.

The deployment typically completes in 3–5 minutes.

3. Force a Runtime Container Refresh

AgentCore may continue using cached container instances after deployment. To ensure the updated container image is used, update the runtime using the AWS CLI.

```
NEW_TAG=$(aws ecr describe-images \
--repository-name motadatamsp/<agent_name> \
--region ap-south-1 --profile motadata \
```



```
--query 'imageDetails | sort_by(@, &imagePushedAt) |
[-1].imageTags[0]' \
--output text)
aws bedrock-agentcore-control update-agent-runtime \
--agent-runtime-id "<runtime_id>" \
--agent-runtime-artifact
"{\"containerConfiguration\":{\"containerUri\":\"001961766007.dkr
.ecr.ap-south-1.amazonaws.com/motadatamsp/<agent_name>:
${NEW_TAG}\"}}\" \
--role-arn "<role_arn>" \
--network-configuration '{"networkMode":"PUBLIC"}' \
--environment-variables '{"FORCE_RESTART":"$(date +%s)"}' \
--region ap-south-1 --profile motadata
```

Replace the following placeholders with the appropriate values:

- <agent_name>
- <runtime_id>
- <role_arn>

4. Verify the Deployment

```
agentcore invoke \
--runtime supervisor_langgraph \
--prompt "Hello" \
--json \
--timeout 60
```

How to Make Code Changes and Redeploy – Frontend

1. Make Code Changes

Frontend source code is located in the frontend/src/ directory.

Key Frontend Files

- pages/MainAppPage.tsx – Main chat interface
- pages/SignInPage.tsx – Login page
- components/ – UI components (NavigationPanel, MessageDisplay, ConversationHistory, etc.)
- store/chatStore.ts – Zustand state management
- services/api/apiClient.ts – Backend API client

Make the required changes and save the files.

2. Build the Frontend

```
cd frontend
npm run build
```



3. Deploy to Amazon S3

```
aws s3 sync dist/ \  
s3://pocassistantfrontendstack-frontends3bucket834ef3a-2-3fwqppbpvcht/ \  
--delete \  
--region ap-south-1 \  
--profile motadata
```

4. Invalidate the CloudFront Cache

```
aws cloudfront create-invalidation \  
--distribution-id E2KKBKVV0SMTV0 \  
--paths "/*" \  
--region us-east-1 \  
--profile motadata
```

Allow approximately 30 seconds for the cache invalidation to propagate globally.

Tenant Account Management

Adding a New Tenant

The application supports multiple tenants. Each tenant's AWS account is accessed using STS AssumeRole with temporary credentials.

To add a new tenant:

1. The tenant creates a cross-account IAM role with the ReadOnlyAccess policy and configures a trust relationship that allows AWS account 001961766007 to assume the role using an External ID.
2. Add the tenant using the application's Add Account UI or create the secret manually:

```
aws secretsmanager create-secret \  
--name "msp-credentials/<tenant_name>" \  
--secret-string '  
{"account_id": "<id>", "role_arn": "arn:aws:iam::<id>:role/<role>", "external_id": "<ext_id>", "customer_name": "<name>"}' \  
--region ap-south-1 --profile motadata
```
3. For RCA telemetry access, create the database secret and tag the RDS instance:

```
aws secretsmanager create-secret \  

```



```
--name "rca-db/<tenant_name>" \  
--secret-string  
'{"username":"rcaadmin","password":"...","dbname":"postgres","port":  
"5432"}' \  
--region ap-south-1 --profile motadata  
aws rds add-tags-to-resource \  
--resource-name "<rds-arn>" \  
--tags Key=tenant,Value=<tenant_name>  
Key=purpose,Value=rca-telemetry \  
--region ap-south-1 --profile motadata
```

Credential Auto-Refresh

Tenant STS credentials expire after one hour.

The backend automatically refreshes credentials when they have expired or are within 10 minutes of expiration.

The frontend also performs an automatic refresh during page load by calling:

PUT /accounts/{name}/refresh

for any account displaying an Expired status.

Telemetry Data Management

Loading Demo Scenario Data

The RCA demonstration scenario (E-Commerce Checkout / Cross-NUMA IRQ Stall) is loaded using the dump_scenario_postgres.sh script.

This script generates OpenTelemetry-compatible telemetry data with timestamps relative to the current time, ensuring the anomaly always appears recent during demonstrations.

```
export PATH="/opt/homebrew/Cellar/libpq/18.4/bin:$PATH"
```

```
bash
```

```
/Users/vynkatesh/Downloads/RCA-Scenario/dump_scenario_postgres  
.sh \  

```

```
--host rca-scenario-db.cng0ieo6guoi.ap-south-1.rds.amazonaws.com  
\  

```

```
--db postgres \  

```

```
--user rcaadmin \  

```

```
--password 'RcaP0c2026x' \  

```

```
--tenants "poc_account" \  

```

```
--truncate
```



Each tenant receives:

- 12,120 metric rows
- 286 log rows
- 840 span rows

covering a 60-minute time window.

To load data for multiple tenants:

--tenants "acme,globex,initech"

CloudWatch Logging and AWS X-Ray Tracing

All application containers running on Amazon ECS Fargate, along with all AgentCore runtimes, are integrated with Amazon CloudWatch Logs for centralized logging and monitoring.

ECS Backend Logs

The ECS task definition uses the awslogs log driver to stream container logs to the following CloudWatch log group:

POCAssistantBackendStack-TaskDefbackendLogGroup15A95F50-Y2cgwpa4lbip

The log stream prefixes are:

- poc-backend/ — Backend application
- adot-collector/ — ADOT Collector

AgentCore Runtime Logs

Each AgentCore runtime writes logs to its own CloudWatch Log Group using the naming convention:

/aws/bedrock-agentcore/runtimes/{runtime_id}-DEFAULT

Log streams are created for each invocation session using the format:

[runtime-logs]{session-uuid}

AWS X-Ray Distributed Tracing

AWS X-Ray traces are collected using the ADOT sidecar container and are available in the CloudWatch X-Ray console.

Each chat request generates a distributed trace showing the complete request flow, including:

- API request latency
- Amazon Bedrock Runtime LLM calls
- DynamoDB operations
- AgentCore Memory interactions



The screenshot shows the AWS WAF console for a Web ACL named 'poc-waf'. The left sidebar contains navigation links for AWS WAF, AI traffic monetization, AWS Shield, AWS Shield network security director, and AWS Firewall Manager. The main content area shows the 'Rules' tab for the 'poc-waf' Web ACL. It lists 3 rules with columns for Name, Action, Priority, and Custom response. Below the rules table, it shows the 'Web ACL capacity units (WCUs) used by your web ACL' section, indicating a total of 925/5000 WCUs. The 'Default web ACL action for requests that don't match any rules' section shows an 'Allow' action. The 'Web ACL CAPTCHA configuration' section is also visible at the bottom.

Accessing CloudWatch Logs

1. Open the AWS Management Console.
2. Navigate to CloudWatch (Region: ap-south-1).
3. Select Logs → Log Groups.
4. Open the appropriate log group.
5. Use the following filter pattern to exclude health check requests:
-"GET /health"
or
-"GET /ping"

CloudWatch logs can be used to:

- Monitor the chat request and response lifecycle.
- Debug agent invocation failures and throttling.
- Track Bedrock model latency and token usage.
- Validate tenant credential propagation.
- Investigate timeout and error conditions.
- Monitor ECS deployment rollouts through task startup logs.